

M11 / PROTECTED ANSWERS

M11 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	5
Open only after a genuine saved attempt	5
DE11-01 Row vs columnar storage; where Parquet fits	5
What strong evidence contains	5
Reasoning to repair	5
Most important failure to catch	5
Fresh retry	5
Do not call Mastered yet if...	5
DE11-02 Parquet schema, row groups and metadata	5
What strong evidence contains	5
Reasoning to repair	6
Most important failure to catch	6
Fresh retry	6
Do not call Mastered yet if...	6
DE11-03 Projection, predicates and analytical reads	6
What strong evidence contains	6
Reasoning to repair	6
Most important failure to catch	6
Fresh retry	6
Do not call Mastered yet if...	6
DE11-04 Compression, small files and compaction	7

What strong evidence contains	7
Reasoning to repair	7
Most important failure to catch	7
Fresh retry	7
Do not call Mastered yet if...	7
DE11-05 Partitioning strategy and over-partitioning	7
What strong evidence contains	7
Reasoning to repair	7
Most important failure to catch	7
Fresh retry	8
Do not call Mastered yet if...	8
DE11-06 Lake vs warehouse vs lakehouse	8
What strong evidence contains	8
Reasoning to repair	8
Most important failure to catch	8
Fresh retry	8
Do not call Mastered yet if...	8
DE11-07 Delta tables, transaction log and ACID concepts	8
What strong evidence contains	8
Reasoning to repair	9
Most important failure to catch	9
Fresh retry	9
Do not call Mastered yet if...	9

DE11-08 MERGE/upsert and current-state records	9
What strong evidence contains	9
Reasoning to repair	9
Most important failure to catch	9
Fresh retry	9
Do not call Mastered yet if...	9
DE11-09 Schema enforcement, controlled evolution and compatibility	10
What strong evidence contains	10
Reasoning to repair	10
Most important failure to catch	10
Fresh retry	10
Do not call Mastered yet if...	10
DE11-10 Time travel, retention, VACUUM safety and medallion checkpoint	10
What strong evidence contains	10
Reasoning to repair	10
Most important failure to catch	10
Fresh retry	11
Do not call Mastered yet if...	11

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a genuine saved attempt

DE11-01 | Row vs columnar storage; where Parquet fits

What strong evidence contains

The simulator reports 8 rows x 6 columns = 48 logical source cells and a two-column projection of 16 logical values. The optional PyArrow lab writes a real Parquet file from the same fixture and prints its schema. Do not claim a performance multiplier from this tiny classroom dataset.

Reasoning to repair

Use Parquet when typed columnar analytical reads, compression and ecosystem interoperability matter. Keep source/raw evidence when the pipeline contract requires it; converting to Parquet is a new representation, not proof that the original source is no longer needed.

Most important failure to catch

The tempting slogan 'Parquet is always faster' is too broad. Reading a single tiny record, ingesting streaming bytes, or exchanging data with a simple external system can favor other formats. The engineering question is workload + access pattern + interoperability, not fashion.

Fresh retry

A report now needs all six columns for only two rows rather than two columns for all rows. Explain why the columnar advantage is less obvious and why file format alone cannot answer the performance question.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-02 | Parquet schema, row groups and metadata

What strong evidence contains

The simulator prints two row groups and identifies which group can be skipped for each supplied predicate. On the learner PC, the real Parquet lab should show 8 rows, 2 row groups (row_group_size=4), schema fields, and per-row-group amount/date statistics when written by the pinned PyArrow runtime.

Reasoning to repair

Treat row-group statistics as observable evidence used by a reader/planner, not as a guarantee. Inspect real file metadata when diagnosing scan behavior and keep the logical predicate separate from the physical skipping opportunity.

Most important failure to catch

Min/max metadata is not a magical index. If each row group contains the full value range, min/max cannot exclude anything. Statistics can also be absent or affected by writer/format details. Never promise data skipping from a file extension alone.

Fresh retry

Change the amount predicate to `amount >= 90`. Re-evaluate both row groups and explain why a less selective boundary changes skipping even though the physical file did not change.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-03 | Projection, predicates and analytical reads

What strong evidence contains

The simulator separates `columns_requested` from `row_groups_candidate`. For the date predicate, RG0 is excluded and RG1 remains. The optional PyArrow read returns only the requested output columns while the filter behavior is inspected separately.

Reasoning to repair

Design schemas and file layout so common analytical filters have useful metadata and common projections avoid wide reads. Verify with engine evidence; SQL syntax expresses intent but does not by itself prove physical I/O.

Most important failure to catch

A common wrong proof is timing one tiny local query and concluding predicate pushdown works. OS cache, tiny data, engine overhead and different physical layout can dominate. Better evidence is metadata, query plan/scan metrics on a capable engine, and controlled comparisons.

Fresh retry

Remove the date predicate but keep only two projected columns. Explain what optimization remains and what optimization disappears.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-04 | Compression, small files and compaction

What strong evidence contains

The planner reports 6 source files / 65 MB and one proposed compacted group. Logical row/amount controls must remain identical before and after the plan. The plan changes physical layout, not business content.

Reasoning to repair

Measure file count, typical file size, partition distribution and query behavior before tuning. Prefer platform-managed optimization where appropriate, but understand the invariant: physical rewrites must preserve logical rows/keys/totals.

Most important failure to catch

Do not set one universal 'perfect file size' and cargo-cult it across engines/workloads. Target sizes, automatic tuning and clustering features vary. Also do not call deleting files 'compaction' - compaction is a rewrite preserving logical data.

Fresh retry

Assume the table is partitioned by day and the small files are spread across 30 daily partitions. Explain why compaction may be constrained within partition boundaries and why over-partitioning can preserve the small-file problem.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-05 | Partitioning strategy and over-partitioning

What strong evidence contains

The simulator shows 4 date partitions, 4 region partitions and 8 order_id partitions for only 8 rows. order_id therefore has one row/partition in the fixture. You should be able to explain why that pattern scales badly.

Reasoning to repair

Partitioning is a physical optimization contract. Keep partition cardinality and files-per-partition observable; do not let an old layout silently become a new bottleneck as volume and query patterns change.

Most important failure to catch

Do not choose a partition key only because users filter it sometimes. Estimate distinct values, data per partition, filter selectivity, write pattern and maintenance behavior. A high-cardinality timestamp can create vast numbers of tiny directories/files.

Fresh retry

The table grows from 50 GB to 5 TB and common filters move from region to date ranges. Explain why a physical-layout decision should be revisited instead of treated as permanent truth.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-06 | Lake vs warehouse vs lakehouse

What strong evidence contains

The simulator prints the same data flow under three architecture lenses and asks which layer owns schema, transactions and serving contracts. There is no score for choosing a fashionable label; the answer must state storage, table semantics and consumers.

Reasoning to repair

Choose architecture by guarantees, workloads, interoperability, cost and operating skill. Keep raw evidence, curated contracts and serving models distinct even when one platform exposes them through a unified UI.

Most important failure to catch

Do not claim 'lakehouse replaces every warehouse' or 'lake means ungoverned mess.' Governance and modeling depend on implementation. Also do not call a folder of Parquet files a transactional table merely because a SQL engine can query them.

Fresh retry

Assume regulations require immutable raw evidence and strong audited serving tables. Explain how a hybrid architecture might intentionally keep both raw lake storage and governed warehouse/lakehouse tables.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-07 | Delta tables, transaction log and ACID concepts

What strong evidence contains

History shows version 0 INIT and version 1 MERGE. Current state has 9 unique order IDs and total amount 1250. Version 0 remains queryable inside the simulator until cleanup rules remove the old snapshot fixture.

Reasoning to repair

Treat the transaction log as table metadata/state, not optional bookkeeping. Use supported table APIs/commands so file rewrites, metadata and versions stay coherent.

Most important failure to catch

Never edit Parquet files or `_delta_log` internals by hand as a normal repair strategy. A table format's guarantees depend on coordinated log + data-file semantics. Random file deletion can corrupt the table or destroy time-travel capability.

Fresh retry

Two writers start from version 1 and both want to update the same business keys. Explain why a transaction protocol needs conflict detection/validation instead of letting both silently overwrite directory contents.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-08 | MERGE/upsert and current-state records

What strong evidence contains

Clean MERGE simulation: 2 updates + 1 insert, 9 unique keys, total 1250. Duplicate-source scenario: rejected before target mutation, current version/total unchanged.

Reasoning to repair

Make MERGE deterministic at the source boundary and idempotent for reruns. Reconcile affected keys/counts/totals and keep change-event evidence when the business requires history.

Most important failure to catch

MERGE is not a license to skip source-grain checks. A source with duplicate business keys can be ambiguous and may error or produce wrong current-state logic depending on engine/rules. DISTINCT is not a universal repair because two different updates can remain genuinely conflicting.

Fresh retry

Add a delete event to the source. Explain why current-state deletion policy and historical/audit requirements must be decided before adding a `WHEN MATCHED . . . DELETE` branch.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-09 | Schema enforcement, controlled evolution and compatibility

What strong evidence contains

Approved additive evolution creates version 2 with `source_system` present and 9 keys / total 1250. Incompatible numeric-type input is rejected; history does not gain a successful commit for that bad write.

Reasoning to repair

Version schema contracts. Prefer explicit, reviewed evolution for important tables. Test downstream compatibility and keep rescue/quarantine paths for unexpected producer changes rather than silently coercing them into trusted data.

Most important failure to catch

A new nullable column can be low-risk; renaming a field used by downstream jobs is not. Some platforms support automatic evolution for specific change classes, but support does not mean the business meaning is safe. Schema compatibility is both technical and contractual.

Fresh retry

Assume a BI report selects columns by name and a streaming reader keeps a tracked schema. Explain why rename/drop planning must include consumers and not only the writer that evolves the table.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.

DE11-10 | Time travel, retention, VACUUM safety and medallion checkpoint

What strong evidence contains

Version 0: 8 keys, total 1100. Current: 9 keys, total 1250. Dry run lists simulated stale files and explicitly reports `deleted=false`. No fixture is physically removed by the classroom command.

Reasoning to repair

Make retention a business/recovery policy, not a random storage tweak. Use table-supported maintenance commands, preserve evidence of changes, and connect Bronze/Silver/Gold layers with explicit contracts and reconciliation rather than names alone.

Most important failure to catch

Do not treat VACUUM as a performance command that can be run with arbitrarily tiny retention. Removing old files can permanently eliminate time-travel/recovery capability and can be dangerous for long-running jobs. Also distinguish OPTIMIZE (rewrite layout) from VACUUM (remove eligible unused files).

Fresh retry

Finance requires 30 days of reproducible month-close recovery instead of 7. Explain which retention decision changes and why the storage-cost trade-off is intentional rather than an error.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with the review closed, or a native runtime proof required by your route is still unexecuted. Simulator evidence proves reasoning and controls, not an engine it did not run.